```
# ============================================================================
# Baltimore Civic Insights Engine (BCIE) — Python / Colab Edition
# Mid-Term Capstone: Multi-Paradigmatic Data Processing Engine
# ============================================================================

import os, io, csv, time, math, urllib.request, concurrent.futures as cf
from collections import defaultdict, Counter
from datetime import datetime

# ----------------------------------------------------------------------------
# CONFIG — put real Open Baltimore CSV URLs here if you have them.
# If the download fails, the script falls back to synthetic sample data
# so the pipeline and analyses are fully demonstrable.
# ----------------------------------------------------------------------------
URL_311 = ""      # e.g. "https://data.baltimorecity.gov/api/views/xxx/rows.csv?accessType=DOWNLOAD"
URL_LIQUOR = ""   # e.g. "https://data.baltimorecity.gov/api/views/yyy/rows.csv?accessType=DOWNLOAD"

# ----------------------------------------------------------------------------
# MODEL LAYER
# ----------------------------------------------------------------------------
class Service311:
    __slots__ = ("service_type", "zip", "neighborhood", "created", "closed")
    def __init__(self, service_type, zip_, neighborhood, created, closed):
        self.service_type = service_type
        self.zip = zip_
        self.neighborhood = neighborhood
        self.created = created
        self.closed = closed

class Liquor:
    __slots__ = ("name", "zip", "neighborhood")
    def __init__(self, name, zip_, neighborhood):
        self.name = name
        self.zip = zip_
        self.neighborhood = neighborhood

class JoinedRecord:
    __slots__ = ("zip", "is_liquor", "service_type", "neighborhood",
                 "created", "closed", "liquor_name")
    def __init__(self, zip_, is_liquor, service_type, neighborhood,
                 created, closed, liquor_name=""):
        self.zip = zip_
        self.is_liquor = is_liquor
        self.service_type = service_type
        self.neighborhood = neighborhood
        self.created = created
        self.closed = closed
        self.liquor_name = liquor_name

# ----------------------------------------------------------------------------
# UTILITIES
# ----------------------------------------------------------------------------
def parse_time_flexible(s):
    s = (s or "").strip()
    if not s:
        return None
    layouts = [
        "%Y-%m-%dT%H:%M:%S.%f",
        "%Y-%m-%dT%H:%M:%S",
        "%Y-%m-%d %H:%M:%S",
        "%m/%d/%Y %I:%M:%S %p",
        "%m/%d/%Y %H:%M:%S",
        "%m/%d/%Y",
        "%Y-%m-%d",
    ]
    for l in layouts:
        try:
            return datetime.strptime(s, l)
        except ValueError:
            continue
    return None

def normalize_zip(s):
    s = (s or "").strip()
```

```
    if not s:
        return ""
    if len(s) >= 5 and s[:5].isdigit():
        return s[:5]
    return s

def find_column(header, *keywords):
    for i, h in enumerate(header):
        hl = (h or "").strip().lower()
        for k in keywords:
            if k.lower() in hl:
                return i
    return -1

def safe_get(rec, idx):
    if idx < 0 or idx >= len(rec):
        return ""
    return (rec[idx] or "").strip()

# ---------------------------------------------------------------------------
# LOADER LAYER (Factory + Strategy pattern)
# ---------------------------------------------------------------------------
class Loader:
    def load(self, src): raise NotImplementedError

class FileLoader(Loader):
    def load(self, src):
        with open(src, "r", encoding="utf-8", errors="replace") as f:
            return f.read()

class HTTPLoader(Loader):
    def load(self, src):
        req = urllib.request.Request(src, headers={"User-Agent": "BCIE/1.0"})
        with urllib.request.urlopen(req, timeout=60) as r:
            return r.read().decode("utf-8", errors="replace")

def new_loader(src):
    if not src:
        return None
    if src.startswith("http://") or src.startswith("https://"):
        return HTTPLoader()
    return FileLoader()

# ---------------------------------------------------------------------------
# PIPELINE — worker-pool CSV parser
# ---------------------------------------------------------------------------
def parse_csv_worker_pool(text, decoder, workers=8):
    """Stream CSV rows through a ThreadPoolExecutor, decoding concurrently."""
    reader = csv.reader(io.StringIO(text))
    try:
        header = next(reader)
    except StopIteration:
        return []
    header = [(h or "").strip() for h in header]

    rows = list(reader)  # materialize rows for parallel decode
    results = []
    with cf.ThreadPoolExecutor(max_workers=workers) as ex:
        futures = [ex.submit(decoder, header, r) for r in rows]
        for fut in cf.as_completed(futures):
            try:
                v = fut.result()
            except Exception:
                continue
            if v is not None:
                results.append(v)
    return results

# ---------------------------------------------------------------------------
# JOIN — concurrent hash join on ZIP
# ---------------------------------------------------------------------------
def join_by_zip(liquor_list, svc_list, workers=8):
    index = defaultdict(list)
    for l in liquor_list:
        if l.zip:
            index[l.zip].append(l)
```

```
    joined = []
    def probe(s):
        if not s.zip:
            return []
        matches = index.get(s.zip, [])
        if not matches:
            return [JoinedRecord(s.zip, False, s.service_type, s.neighborhood,
                                 s.created, s.closed)]
        return [JoinedRecord(s.zip, True, s.service_type, s.neighborhood,
                             s.created, s.closed, l.name) for l in matches]

    with cf.ThreadPoolExecutor(max_workers=workers) as ex:
        for chunk in ex.map(probe, svc_list):
            joined.extend(chunk)
    return joined

# ---------------------------------------------------------------------------
# ANALYSIS — Strategy pattern
# ---------------------------------------------------------------------------
class Result:
    def __init__(self, title, headers=None, rows=None, chart="", narrative=""):
        self.title = title
        self.headers = headers or []
        self.rows = rows or []
        self.chart = chart
        self.narrative = narrative

def bar_chart(title, bars, width=40):
    if not bars:
        return title + "\n  (no data)\n"
    mx = max(v for _, v in bars) or 1.0
    lines = [title]
    for label, val in bars:
        n = int(val / mx * width)
        lines.append(f"  {label:<14} |{'█' * n} {val:.2f}")
    return "\n".join(lines) + "\n"

def top_n_by_value(m, n=10):
    return sorted(m.items(), key=lambda kv: kv[1], reverse=True)[:n]

# --- Q1: 311 Hotspots ------------------------------------------------------
def q1_hotspots(data):
    counts = Counter()
    for r in data:
        if r.service_type:
            counts[r.zip] += 1
    bars = top_n_by_value(counts, 10)
    rows = [[z, str(int(v))] for z, v in bars]
    return Result(
        title="Q1: Top 10 ZIP Codes by 311 Request Volume",
        headers=["ZIP", "Requests"],
        rows=rows,
        chart=bar_chart("311 Requests by ZIP (Top 10)", bars),
        narrative="These ZIP codes generate the highest absolute 311 demand. "
                  "Allocate intake capacity and field crews proportionally."
    )

# --- Q2: Response time -----------------------------------------------------
def q2_response_time(data):
    sum_h, cnt = defaultdict(float), Counter()
    for r in data:
        if r.created and r.closed:
            d = (r.closed - r.created).total_seconds() / 3600.0
            if d >= 0:
                sum_h[r.zip] += d
                cnt[r.zip] += 1
    avg = {z: sum_h[z] / cnt[z] for z in sum_h if cnt[z]}
    bars = top_n_by_value(avg, 10)
    rows = [[z, f"{v:.1f}", str(cnt[z])] for z, v in bars]
    return Result(
        title="Q2: Slowest ZIP Codes for 311 Closure (hours)",
        headers=["ZIP", "Avg Hours", "Requests"],
        rows=rows,
        chart=bar_chart("Avg Closure Time (hours) — Top 10 Slowest", bars),
        narrative="ZIP codes with the longest average closure times need process "
```

```
                "review or additional staffing. Cross-check with Q1 volume."
    )

# --- Q3: Liquor vs noise ----------------------------------------------------
def q3_liquor_noise(data):
    agg = defaultdict(lambda: {"liquor": 0, "noise": 0})
    for r in data:
        a = agg[r.zip]
        if r.is_liquor:
            a["liquor"] += 1
        if "noise" in (r.service_type or "").lower():
            a["noise"] += 1
    ratio = {z: a["noise"] / a["liquor"] for z, a in agg.items() if a["liquor"] > 0}
    bars = top_n_by_value(ratio, 10)
    rows = [[z, str(agg[z]["noise"]), str(agg[z]["liquor"]), f"{v:.2f}"]
            for z, v in bars]
    return Result(
        title="Q3: Noise Complaints per Liquor License (Top 10)",
        headers=["ZIP", "Noise", "Liquor", "Ratio"],
        rows=rows,
        chart=bar_chart("Noise Complaints per Liquor License", bars),
        narrative="High ratios flag ZIP codes where liquor density alone doesn't "
                  "explain noise — investigate specific venues."
    )

# --- Q4: Seasonal -----------------------------------------------------------
def q4_seasonal(data):
    months = Counter()
    for r in data:
        if r.created:
            months[r.created.strftime("%Y-%m")] += 1
    keys = sorted(months.keys())[-12:]
    bars = [(k, months[k]) for k in keys]
    rows = [[k, str(months[k])] for k in keys]
    return Result(
        title="Q4: 311 Volume by Month (last 12)",
        headers=["Month", "Requests"],
        rows=rows,
        chart=bar_chart("Monthly 311 Volume", bars),
        narrative="Seasonal peaks inform surge staffing and preventive maintenance."
    )

# --- Q5: Equity -------------------------------------------------------------
def q5_equity(data):
    agg = defaultdict(lambda: {"liquor": 0, "sum": 0.0, "cnt": 0})
    for r in data:
        a = agg[r.zip]
        if r.is_liquor:
            a["liquor"] += 1
        if r.created and r.closed:
            d = (r.closed - r.created).total_seconds() / 3600.0
            if d >= 0:
                a["sum"] += d
                a["cnt"] += 1
    stats = []
    for z, a in agg.items():
        if a["cnt"] > 0:
            stats.append((z, a["liquor"], a["sum"] / a["cnt"]))
    stats.sort(key=lambda t: t[1])
    if not stats:
        return Result(title="Q5: Equity Analysis (no data)")
    mid = len(stats) // 2
    low = stats[:mid]
    high = stats[mid:]
    low_avg = sum(s[2] for s in low) / len(low) if low else 0
    high_avg = sum(s[2] for s in high) / len(high) if high else 0
    bars = [("Low Liquor", low_avg), ("High Liquor", high_avg)]
    return Result(
        title="Q5: Avg 311 Closure Time — Low vs High Liquor-Density ZIPs",
        headers=["Group", "ZIPs", "Avg Hours"],
        rows=[
            ["Low Liquor Density", str(len(low)), f"{low_avg:.1f}"],
            ["High Liquor Density", str(len(high)), f"{high_avg:.1f}"],
        ],
        chart=bar_chart("Avg Closure Time (hours)", bars),
        narrative="If high-liquor ZIPs close faster, low-density neighborhoods may "
```

```
                "be underserved. Investigate service equity."
    )

# ---------------------------------------------------------------------------
# RENDER
# ---------------------------------------------------------------------------
def print_results(results):
    for r in results:
        print("\n" + "=" * 60)
        print("  " + r.title)
        print("=" * 60)
        if r.chart:
            print(r.chart)
        if r.rows:
            print("\t".join(r.headers))
            for row in r.rows:
                print("\t".join(row))
        if r.narrative:
            print("\n>> " + r.narrative)
    print()

# ---------------------------------------------------------------------------
# SAMPLE DATA (fallback so the pipeline is always demonstrable)
# ---------------------------------------------------------------------------
SAMPLE_311_CSV = """ServiceType,ZipCode,Neighborhood,CreatedDate,ClosedDate
Noise Complaint,21230,Federal Hill,2024-01-15 10:00:00,2024-01-17 14:00:00
Noise Complaint,21230,Federal Hill,2024-02-02 22:30:00,2024-02-05 08:00:00
Noise Complaint,21201,Downtown,2024-01-20 23:00:00,2024-01-22 09:00:00
Trash Collection,21201,Downtown,2024-03-01 08:00:00,2024-03-02 08:00:00
Trash Collection,21205,East Baltimore,2024-03-05 09:00:00,2024-03-12 09:00:00
Pothole,21205,East Baltimore,2024-04-01 07:00:00,2024-04-20 17:00:00
Pothole,21207,Forest Park,2024-04-10 07:00:00,2024-05-01 17:00:00
Street Light Out,21207,Forest Park,2024-05-02 19:00:00,2024-05-20 12:00:00
Noise Complaint,21202,Jonestown,2024-06-15 23:00:00,2024-06-16 10:00:00
Noise Complaint,21202,Jonestown,2024-07-04 23:30:00,2024-07-06 11:00:00
Trash Collection,21230,Federal Hill,2024-07-10 08:00:00,2024-07-11 08:00:00
Pothole,21230,Federal Hill,2024-08-01 07:00:00,2024-08-15 17:00:00
Noise Complaint,21205,East Baltimore,2024-09-01 22:00:00,2024-09-02 09:00:00
Trash Collection,21202,Jonestown,2024-10-01 08:00:00,2024-10-03 08:00:00
Street Light Out,21201,Downtown,2024-11-01 19:00:00,2024-11-15 12:00:00
Pothole,21201,Downtown,2024-12-01 07:00:00,2024-12-20 17:00:00
"""

SAMPLE_LIQUOR_CSV = """Name,ZipCode,Neighborhood
The Federal Tap,21230,Federal Hill
Harbor Bar,21230,Federal Hill
Downtown Lounge,21201,Downtown
City Spirits,21201,Downtown
East Side Pub,21205,East Baltimore
Jonestown Ale House,21202,Jonestown
Forest Park Grill,21207,Forest Park
"""

# ---------------------------------------------------------------------------
# MAIN
# ---------------------------------------------------------------------------
def main():
    print("=" * 60)
    print("  Baltimore Civic Insights Engine (BCIE)")
    print("=" * 60)

    # --- Ingestion (Factory) ---
    def fetch(url, fallback):
        loader = new_loader(url)
        if loader is None:
            print(f"[loader] no URL configured — using sample data")
            return fallback
        try:
            print(f"[loader] downloading {url[:70]}...")
            text = loader.load(url)
            print(f"[loader] downloaded {len(text):,} bytes")
            return text
        except Exception as e:
            print(f"[loader] download failed ({e}) — using sample data")
            return fallback
```

```
    svc_text = fetch(URL_311, SAMPLE_311_CSV)
    liq_text = fetch(URL_LIQUOR, SAMPLE_LIQUOR_CSV)

    # --- Parse (worker pools) ---
    def decode_311(header, rec):
        s = Service311(
            service_type=safe_get(rec, find_column(header, "servicetype", "service type", "type", "srtype", "requesttype")),
            zip_=normalize_zip(safe_get(rec, find_column(header, "zip", "zipcode", "postal"))),
            neighborhood=safe_get(rec, find_column(header, "neighborhood", "community")),
            created=parse_time_flexible(safe_get(rec, find_column(header, "createddate", "created", "datecreated"))),
            closed=parse_time_flexible(safe_get(rec, find_column(header, "closeddate", "closed", "dateclosed"))),
        )
        if not s.zip and not s.service_type:
            return None
        return s

    def decode_liquor(header, rec):
        l = Liquor(
            name=safe_get(rec, find_column(header, "name", "establishment", "business", "licensee")),
            zip_=normalize_zip(safe_get(rec, find_column(header, "zip", "zipcode", "postal"))),
            neighborhood=safe_get(rec, find_column(header, "neighborhood", "community")),
        )
        if not l.zip:
            return None
        return l

    workers = min(8, (os.cpu_count() or 4))
    print(f"[parse] decoding with {workers} workers...")

    with cf.ThreadPoolExecutor(max_workers=2) as ex:
        f_svc = ex.submit(parse_csv_worker_pool, svc_text, decode_311, workers)
        f_liq = ex.submit(parse_csv_worker_pool, liq_text, decode_liquor, workers)
        svc_list = f_svc.result()
        liquor_list = f_liq.result()

    print(f"[parse] 311 rows: {len(svc_list):,}  |  liquor rows: {len(liquor_list):,}")

    # --- Join (concurrent hash join on ZIP) ---
    joined = join_by_zip(liquor_list, svc_list, workers)
    print(f"[join] joined records: {len(joined):,}")

    # --- Analysis (Strategy pattern, run concurrently) ---
    strategies = [q1_hotspots, q2_response_time, q3_liquor_noise, q4_seasonal, q5_equity]
    results = [None] * len(strategies)
    with cf.ThreadPoolExecutor(max_workers=len(strategies)) as ex:
        futures = {ex.submit(s, joined): i for i, s in enumerate(strategies)}
        for fut in cf.as_completed(futures):
            i = futures[fut]
            try:
                results[i] = fut.result()
            except Exception as e:
                results[i] = Result(title=f"{strategies[i].__name__} failed: {e}")

    # --- Output ---
    print_results(results)

if __name__ == "__main__":
    main()
```

```
============================================================
  Baltimore Civic Insights Engine (BCIE)
============================================================
[loader] no URL configured — using sample data
[loader] no URL configured — using sample data
[parse] decoding with 2 workers...
[parse] 311 rows: 16  |  liquor rows: 7
[join] joined records: 24

============================================================
  Q1: Top 10 ZIP Codes by 311 Request Volume
============================================================
311 Requests by ZIP (Top 10)
  21230          |████████████████████████████████ 8.00
  21201          |████████████████████████████████ 8.00
  21205          |████████████ 3.00
  21202          |████████████ 3.00
  21207          |█████████ 2.00
```

```
ZIP     Requests
21230   8
21201   8
21205   3
21202   3
21207   2

>> These ZIP codes generate the highest absolute 311 demand. Allocate intake capacity and field crews proportionally.

============================================================
  Q2: Slowest ZIP Codes for 311 Closure (hours)
============================================================
Avg Closure Time (hours) — Top 10 Slowest
  21207          |████████████████████████████████ 469.50
  21205          |███████████████ 215.00
  21201          |███████████████ 213.25
  21230          |████████ 119.88
  21202          |██ 31.50

ZIP     Avg Hours       Requests
21207   469.5   2
21205   215.0   3
21201   213.2   8
21230   119.9   8
21202   31.5    3

>> ZIP codes with the longest average closure times need process review or additional staffing. Cross-check with Q1 volume.

============================================================
  Q3: Noise Complaints per Liquor License (Top 10)
============================================================
Noise Complaints per Liquor License
  21202          |████████████████████████████████ 0.67
  21230          |████████████████████████ 0.50
  21205          |████████████████ 0.33
  21201          |████████████ 0.25
  21207          | 0.00

ZIP     Noise   Liquor  Ratio
```